

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the Data Labeling Platform. This document follows the IEEE 29148 standard. It will illustrate the purpose and complete declaration for the development of the system. It explains system constraints, interface requirements, and interactions with external entities.

1.2 Scope

The **Data Labeling Platform** is a web-based system designed to facilitate and manage the process of annotating data (primarily images) for Machine Learning and AI projects. The system supports full project management, role-based access control (RBAC), data item tracking, manual annotation (bounding boxes using YOLO format), review workflows (approval/rejection), and data export capabilities.

Key objectives include:

- Centralized management of annotation projects.
- Efficient allocation of data to Annotators.
- Quality assurance through a Reviewer role.
- Secure storage of image data using Cloudinary.
- Seamless authentication via Google OAuth2.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS**: Software Requirements Specification
- **RBAC**: Role-Based Access Control
- **JWT**: JSON Web Token
- **API**: Application Programming Interface
- **YOLO**: You Only Look Once (A format for object detection bounding boxes: xCenter, yCenter, width, height)
- **OAUTH2**: Open Authorization protocol
- **ER**: Entity-Relationship
- **UML**: Unified Modeling Language

1.4 References

- IEEE Std 29148-2018: Systems and software engineering — Life cycle processes — Requirements engineering.
- Project Source Code Repository: Spring Boot 4.0.5, MySQL 8.
- OAuth 2.0 Authorization Framework (RFC 6749).

1.5 Document Overview

The remainder of this document is organized as follows:

- **Section 2** provides a general overview of the system, including stakeholders, user roles, constraints, and assumptions.
 - **Section 3** specifies detailed requirements: functional requirements, non-functional requirements, data requirements (data dictionary), and API specifications.
 - **Section 4** presents UML diagrams representing system design and workflow.
 - **Section 5** contains the Requirement Traceability Matrix (RTM).
-

2. Overall Description

2.1 System Perspective

The Data Labeling Platform is an independent, self-contained web application operating on a client-server architecture.

- **Backend:** Built with Java Spring Boot, exposing RESTful APIs.
- **Database:** MySQL 8 used for relational data persistence.
- **Storage:** External integration with Cloudinary for handling and serving image uploads.
- **Frontend:** A React/Next.js based UI (implied in `/data-labeling ui` folder).

2.2 System Functions

The system provides the following major functions:

- **Authentication & Security:** Local login, Google OAuth2, JWT generation and validation, token blacklisting.
- **User Management:** Registration, profile updates, role and permission assignment.
- **Project Management:** Creating projects, assigning managers, managing project members (Annotators, Reviewers), and dashboard analytics.
- **Data Item Management:** Uploading data files, assigning labels, tracking data status (UNLABELED, LABELED, REVIEWING, APPROVED, REJECTED).
- **Annotation:** Creating and managing annotations (YOLO format bounding boxes) for data items.
- **Review System:** Approving or rejecting annotated data, tracking review logs and reasons.
- **Export & Audit:** Exporting approved datasets for AI training, auditing all system actions for security compliance.
- **System Configuration:** Managing global system settings dynamically (e.g., enabling/disabling registration, maintenance mode).

2.3 User Characteristics

The system targets four primary user groups (Stakeholders):

1. Administrator (ADMIN)

- **Characteristics:** Technical users responsible for system maintenance.
- **Responsibilities:** Manage users, assign roles, monitor audit logs, manage system configurations, full access to all projects.

2. Project Manager (MANAGER)

- **Characteristics:** Operational leaders organizing data labeling campaigns.
- **Responsibilities:** Create projects, define labels, upload data items, assign Annotators and Reviewers to projects, monitor progress via dashboards, and export final datasets.

3. Annotator (ANNOTATOR)

- **Characteristics:** Data entry operators or domain experts who label data.
- **Responsibilities:** View assigned projects, fetch unlabeled/rejected data items, draw bounding boxes (annotations), and submit data for review.

4. Reviewer (REVIEWER)

- **Characteristics:** Senior annotators or Quality Assurance (QA) personnel.

- **Responsibilities:** Inspect submitted annotations, approve correct data, and reject incorrect data with comments (ReviewLogs).

2.4 General Constraints

- **Regulatory:** The system must comply with data privacy policies; user passwords must be hashed (Bcrypt).
- **Hardware Limitations:** Image uploads are constrained by Cloudinary's tier limits.
- **Technology Constraints:** Backend must run on Java 21+ and Spring Boot 4.0.5+.

2.5 Assumptions and Dependencies

- **Assumptions:**
 - Users have modern web browsers (Chrome, Firefox, Edge).
 - Administrators will secure the database credentials and JWT Secret Key.
- **Dependencies:**
 - Depends on Cloudinary API for image storage.
 - Depends on Google OAuth2 API for Single Sign-On (SSO).
 - MySQL Database must be highly available to prevent application crashes.

3. Specific Requirements

3.1 Functional Requirements

This section outlines the detailed functional requirements of the system, complete with Use Cases, Main/Alternative Flows, and Acceptance Criteria.

3.1.1 Authentication & Security Module (FR-01)

Description: The system must provide secure mechanisms for users to authenticate and maintain sessions.

Use Case 1: Local Login

- **Preconditions:** The user has registered an account and is active.
- **Main Flow:**
 1. User enters username and password.
 2. System validates credentials against the database.
 3. System generates a JWT access token and refresh token.
 4. System returns the tokens to the client.
- **Alternative Flow:**
 - (2a) User enters wrong password. System returns 401 Unauthorized.
- **Postconditions:** User is authenticated and can access secured endpoints.
- **Acceptance Criteria:**
 - Passwords must be hashed using Bcrypt.
 - Token must contain User ID, Roles, and Permissions in claims.

Use Case 2: Google OAuth2 Login

- **Preconditions:** User has a valid Google account.
- **Main Flow:**
 1. User clicks "Login with Google".
 2. Client receives Google ID token and sends it to backend `/api/auth/google`.

3. Backend validates the Google token.
4. If user doesn't exist, system auto-registers the user.
5. System generates JWT and returns it.

- **Postconditions:** User is logged in via SSO.

Use Case 3: Logout

- **Main Flow:**
 1. User clicks logout.
 2. Client sends current JWT to `/api/auth/logout`.
 3. System adds the token ID (JTI) to the `invalidated_token` table.
- **Acceptance Criteria:** Blacklisted tokens cannot be used to access APIs, returning 401.

3.1.2 User Management Module (FR-02)

Description: The system allows administrators to manage users and their roles/permissions.

Use Case 4: Manage Users (CRUD)

- **Preconditions:** User has `MANAGE_USER` permission (usually ADMIN).
- **Main Flow:**
 1. Admin requests to view the user list.
 2. Admin can create, update, or delete a user.
 3. Admin can assign `Role` (ADMIN, MANAGER, ANNOTATOR, REVIEWER) to a user.
- **Acceptance Criteria:** Deleting a user with assigned tasks must either cascade delete tasks or restrict deletion.

3.1.3 Project Management Module (FR-03)

Description: Capabilities to organize annotation work into Projects.

Use Case 5: Create and Configure Project

- **Preconditions:** User has `MANAGE_PROJECT` permission.
- **Main Flow:**
 1. User inputs Project Name, Description, and Reviewer ID.
 2. System creates the Project with status `DRAFT`.
 3. Manager adds `Labels` (Name and Color) to the Project.
 4. Manager adds `ProjectMembers` (Users assigned to this project).
- **Postconditions:** A new project is created and ready to receive data.

3.1.4 Dataset Management Module (FR-04)

Description: Uploading and organizing images to be labeled.

Use Case 6: Upload Data Items

- **Preconditions:** Project exists and Manager is authenticated.
- **Main Flow:**
 1. Manager uploads an image file.
 2. System uploads the file to Cloudinary and retrieves the secure `fileUrl`.
 3. System creates a `DataItem` record linked to the Project with status `UNLABELED`.
- **Exception Flow:** Cloudinary API fails -> System aborts database save and returns 500 Error.

3.1.5 Annotation Workflow Module (FR-05)

Description: The core labeling functionality.

Use Case 7: Annotate Data Item

- **Preconditions:** User is ANNOTATOR and is a member of the project.
- **Main Flow:**
 1. Annotator requests the next UNLABELED or REJECTED item in the project.
 2. System returns the Dataltem.
 3. Annotator draws bounding boxes and submits Annotations (label, xCenter, yCenter, width, height).
 4. System saves Annotations, updates Dataltem status to REVIEWING .
- **Alternative Flow:**
 - Annotator skips the item. Status remains unchanged.
- **Acceptance Criteria:** Tightly coupled Business Rule: Annotators can only access projects they are assigned to (Enforce BOLA/IDOR protection).

3.1.6 Review Workflow Module (FR-06)

Description: Quality assurance for annotations.

Use Case 8: Approve/Reject Data

- **Preconditions:** User has REVIEW_DATA permission.
- **Main Flow (Approve):**
 1. Reviewer fetches a REVIEWING data item.
 2. Reviewer clicks Approve.
 3. System changes Dataltem status to APPROVED .
 4. System records action in ReviewLog .
- **Alternative Flow (Reject):**
 1. Reviewer clicks Reject and provides a rejectReason .
 2. System changes status to REJECTED .
 3. System records in ReviewLog .
- **Postconditions:** Item is ready for export (if approved) or re-annotation (if rejected).

3.1.7 Export and Audit Module (FR-07)

Description: Outputting data and tracing activities.

Use Case 9: Export Dataset

- **Preconditions:** User has EXPORT_DATA permission.
- **Main Flow:**
 1. User requests export for a specific Project.
 2. System gathers all APPROVED data items and their annotations.
 3. System formats output as JSON/CSV/YOLO text files.
 4. System returns the file to the client.

Use Case 10: System Audit Logging

- **Description:** Every sensitive API call (Create, Update, Delete) is intercepted (likely via AOP/Interceptor) and a record is created in AuditLog containing username, action, target entity, details, IP address, and

timestamp.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- **Response Time:** API endpoints (excluding file uploads) must respond within 500ms under normal load.
- **Throughput:** The system must handle concurrent annotation submissions from at least 100 users.

3.2.2 Security Requirements

- **Authentication:** All endpoints except `/api/auth/**` must require a valid JWT.
- **Authorization:** Method-level security (e.g., `@PreAuthorize`) must be implemented to check permissions against user roles.
- **Data Protection:** Passwords must not be stored in plain text. JWT Tokens must be signed using HMAC SHA-512 with a minimum 256-bit secret key.

3.2.3 Reliability and Availability

- **Cloud Dependency:** The system depends on Cloudinary for image delivery. If Cloudinary is down, the system should gracefully fail and alert the user.
- **Database Transactions:** Operations spanning multiple tables (e.g., deleting a user and their tokens) must be transactional to prevent orphaned records.

3.2.4 Maintainability

- The backend codebase must follow clean architecture principles, separating Controllers, Services, and Repositories.
- MapStruct should be used for all DTO to Entity conversions to avoid boilerplate code.

3.3 Business Rules

- **BR-01 (Role Exclusivity):** An ADMIN has all permissions implicitly.
- **BR-02 (Data Privacy):** A user cannot view `DataItems` of a `Project` unless they are explicitly added to the `ProjectMember` table or have the `MANAGE_PROJECT` system-wide permission.
- **BR-03 (State Machine):** A `DataItem` must follow the transition: `UNLABELED -> REVIEWING -> (APPROVED or REJECTED)`. `REJECTED -> REVIEWING`. Only `APPROVED` data can be exported.
- **BR-04 (Maintenance Mode):** If `SystemConfig` key `SYSTEM_MAINTENANCE` is set to `true`, only ADMIN users can log in or perform actions.

3.4 Data Requirements & Data Dictionary

This section describes the logical data entities and their attributes required to support the system functionalities.

3.4.1 Data Entities (Data Dictionary)

Table 1: User Stores system user information.

Attribute	Data Type	Description	Key
id	UUID (String)	Unique identifier for the user	PK
username	String	Unique login name	Unique
password	String	Hashed password	-

firstName	String	User's first name	-
lastName	String	User's last name	-
email	String	User's email address	Unique
dateOfBirth	Date	User's date of birth	-

Table 2: Role Stores RBAC roles.

Attribute	Data Type	Description	Key
name	String	Role name (e.g., ADMIN)	PK
description	String	Detailed description	-

Table 3: Permission Stores granular permissions.

Attribute	Data Type	Description	Key
name	String	Permission name (e.g., MANAGE_USER)	PK
description	String	Detailed description	-

Table 4: Project Stores information about annotation projects.

Attribute	Data Type	Description	Key
id	UUID	Unique project identifier	PK
name	String	Name of the project	-
description	Text	Instructions or details	-
status	Enum	ProjectStatus (DRAFT, etc.)	-
manager_id	UUID	User ID of project manager	FK
reviewer_id	UUID	User ID of default reviewer	FK
createdAt	DateTime	Creation timestamp	-
updatedAt	DateTime	Last updated timestamp	-

Table 5: DataItem Stores images uploaded for annotation.

Attribute	Data Type	Description	Key
id	UUID	Unique item identifier	PK
fileName	String	Original file name	-
fileUrl	Text	Cloudinary URL	-
rejectReason	String	Reason for rejection	-

status	Enum	DataltemStatus	-
project_id	UUID	Belongs to project	FK

Table 6: Annotation Stores bounding box labels per Dataltem.

Attribute	Data Type	Description	Key
id	UUID	Unique annotation identifier	PK
label_id	UUID	Linked Label	FK
xCenter	Double	YOLO X coordinate	-
yCenter	Double	YOLO Y coordinate	-
width	Double	YOLO Width	-
height	Double	YOLO Height	-
data_item_id	UUID	Linked Dataltem	FK
user_id	UUID	Annotator who drew this	FK

Table 7: Label Stores valid labels for a project.

Attribute	Data Type	Description	Key
id	UUID	Unique label identifier	PK
name	String	Label text (e.g., "Car")	-
color	String	Hex color (e.g., "#FF0000")	-
project_id	UUID	Belongs to project	FK

Table 8: ReviewLog Stores history of reviewer actions.

Attribute	Data Type	Description	Key
id	UUID	Unique log identifier	PK
data_item_id	UUID	Linked Dataltem	FK
reviewer_id	UUID	Linked User	FK
comment	Text	Reason for action	-
createdAt	DateTime	Timestamp of review	-

Table 9: AuditLog Stores all sensitive system actions.

Attribute	Data Type	Description	Key
id	UUID	Log identifier	PK

username	String	Who performed action	-
action	String	CRUD action	-
targetEntity	String	Affected entity	-
details	String	Extra context	-
ipAddress	String	Source IP	-
status	String	SUCCESS/FAIL	-
timestamp	DateTime	When action occurred	-

Table 10: SystemConfig Dynamic system settings.

Attribute	Data Type	Description	Key
config_key	String	Setting Key (e.g., MAINT)	PK
config_value	Text	Value	-
description	String	Contextual help	-

Table 11: InvalidatedToken Blacklisted JWT tokens from logout.

Attribute	Data Type	Description	Key
id	String	JWT JTI	PK
expiryTime	Date	Expiration of token	-

3.5 External Interface Requirements

3.5.1 User Interfaces

The system provides a web-based User Interface (SPA) built with React/Next.js. It features dashboards for Project Managers, a bounding box Canvas for Annotators, and an approval table for Reviewers.

3.5.2 Hardware Interfaces

No specific hardware interfaces beyond a standard computer monitor for the user interface, with mouse support for drawing accurate annotations.

3.5.3 Software Interfaces

- **Database System:** MySQL 8.0 running on default port 3306.
- **Cloud Storage:** Cloudinary API for image uploads and CDN delivery.
- **Authentication:** Google OAuth2 API endpoints.

3.5.4 Communication Interfaces

- **Protocol:** HTTP/HTTPS over TCP/IP.
- **Data Format:** JSON for all Request/Response bodies in the REST API.

3.6 API Specifications

This section specifies the RESTful endpoints used by the platform.

3.6.1 Authentication API

POST /api/auth/token

- **Description:** Authenticate user and issue JWT.
- **Request Body:** {"username": "string", "password": "string"}
- **Response Body:** {"code": 1000, "result": {"token": "jwt_string", "authenticated": true}}
- **Validation Rules:** Username and password cannot be blank.
- **Error Codes:** 401 Unauthorized (Invalid credentials).

POST /api/auth/google

- **Description:** Login using Google OAuth2.
- **Request Body:** {"token": "google_id_token"}
- **Response Body:** {"code": 1000, "result": {"token": "jwt_string"}}
- **Validation Rules:** Token must be a valid Google-issued JWT.

3.6.2 User API

POST /api/users/register

- **Description:** Create a new user account.
- **Request Body:** {"username": "str", "password": "str", "email": "str"}
- **Response Body:** User profile data without password.
- **Validation Rules:** Username and Email must be unique. Password > 6 chars.
- **Error Codes:** 400 Bad Request (Duplicate username).

GET /api/users/myInfor

- **Description:** Retrieve current authenticated user's info.
- **Header:** Authorization: Bearer <jwt>
- **Response Body:** { "id": "...", "username": "...", "roles": [...] } }

3.6.3 Project API

POST /api/projects

- **Description:** Create a new annotation project.
- **Header:** Authorization: Bearer <jwt>
- **Request Body:** {"name": "Detect Cars", "description": "...", "reviewerId": "..."} }
- **Response Body:** Project entity.
- **Validation Rules:** User must have MANAGE_PROJECT permission.

3.6.4 Annotation API

POST /api/annotations

- **Description:** Submit annotations for a DataItem.
- **Header:** Authorization: Bearer <jwt>
- **Request Body:**

```
{
  "dataItemId": "uuid",
  "annotations": [
    {"labelId": "uuid", "xCenter": 0.5, "yCenter": 0.5, "width": 0.1, "height": 0.1}
  ]
}
```

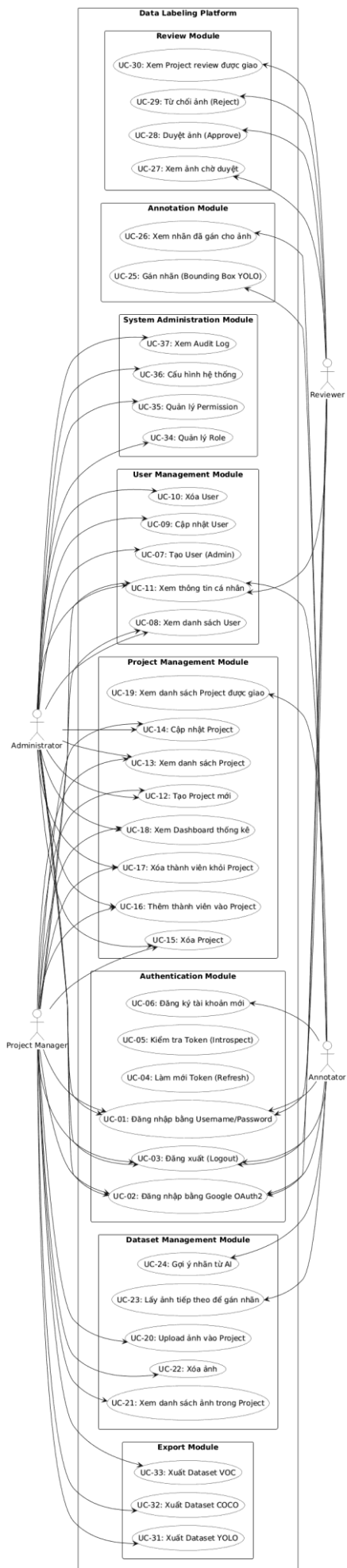
- **Response Body:** Success message.
 - **Validation Rules:** Coordinates must be between 0 and 1. DataItem status must be UNLABELED or REJECTED.
-

4. UML Artifacts (System Models) — Chi tiết theo từng luồng chức năng

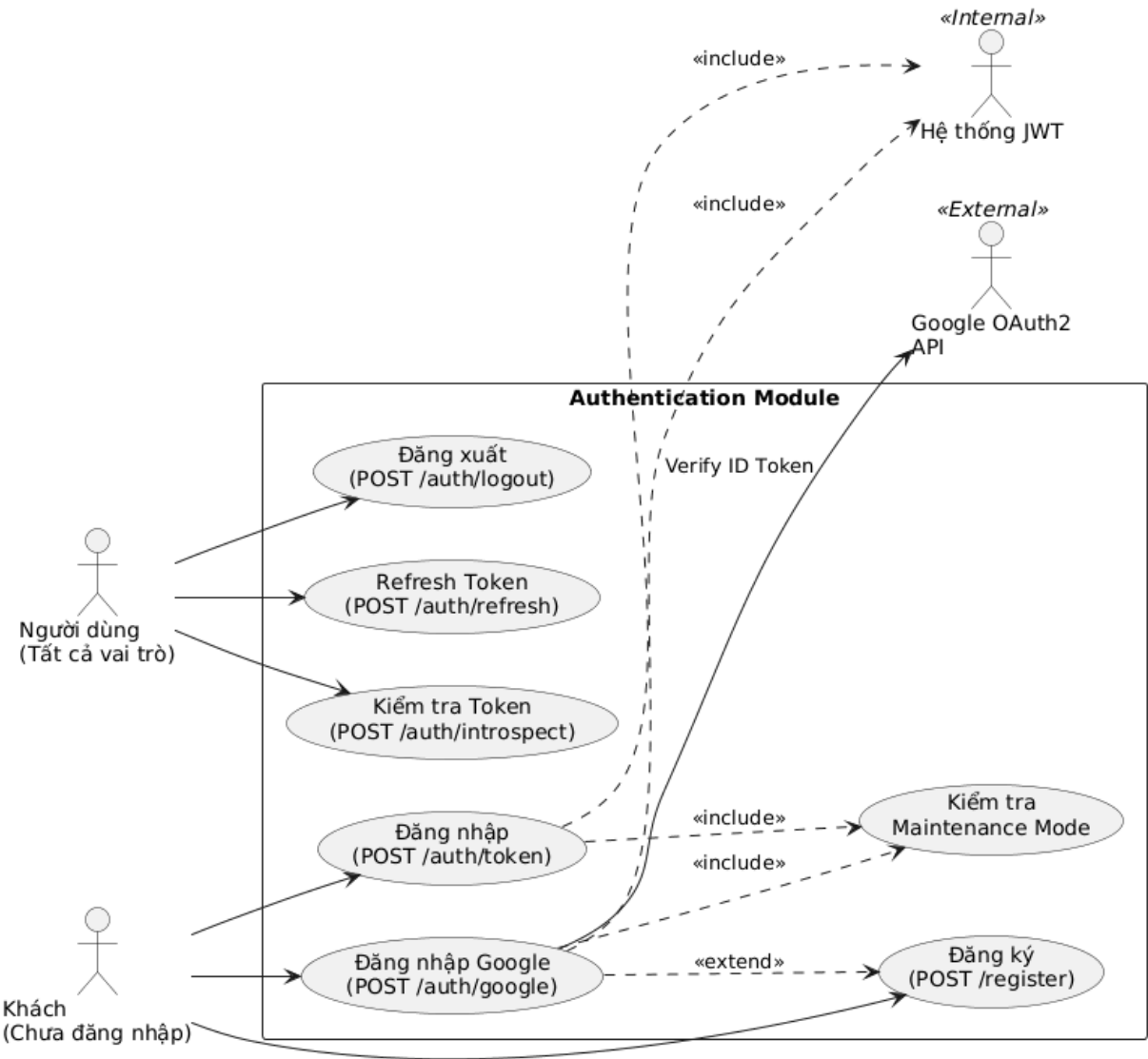
Phần này trình bày các diagram UML chi tiết, **bám sát 100% source code thực tế** của hệ thống. Mỗi diagram được tạo riêng cho từng luồng chức năng để giảng viên có thể đối chiếu trực tiếp với code.

4.1 Use Case Diagrams

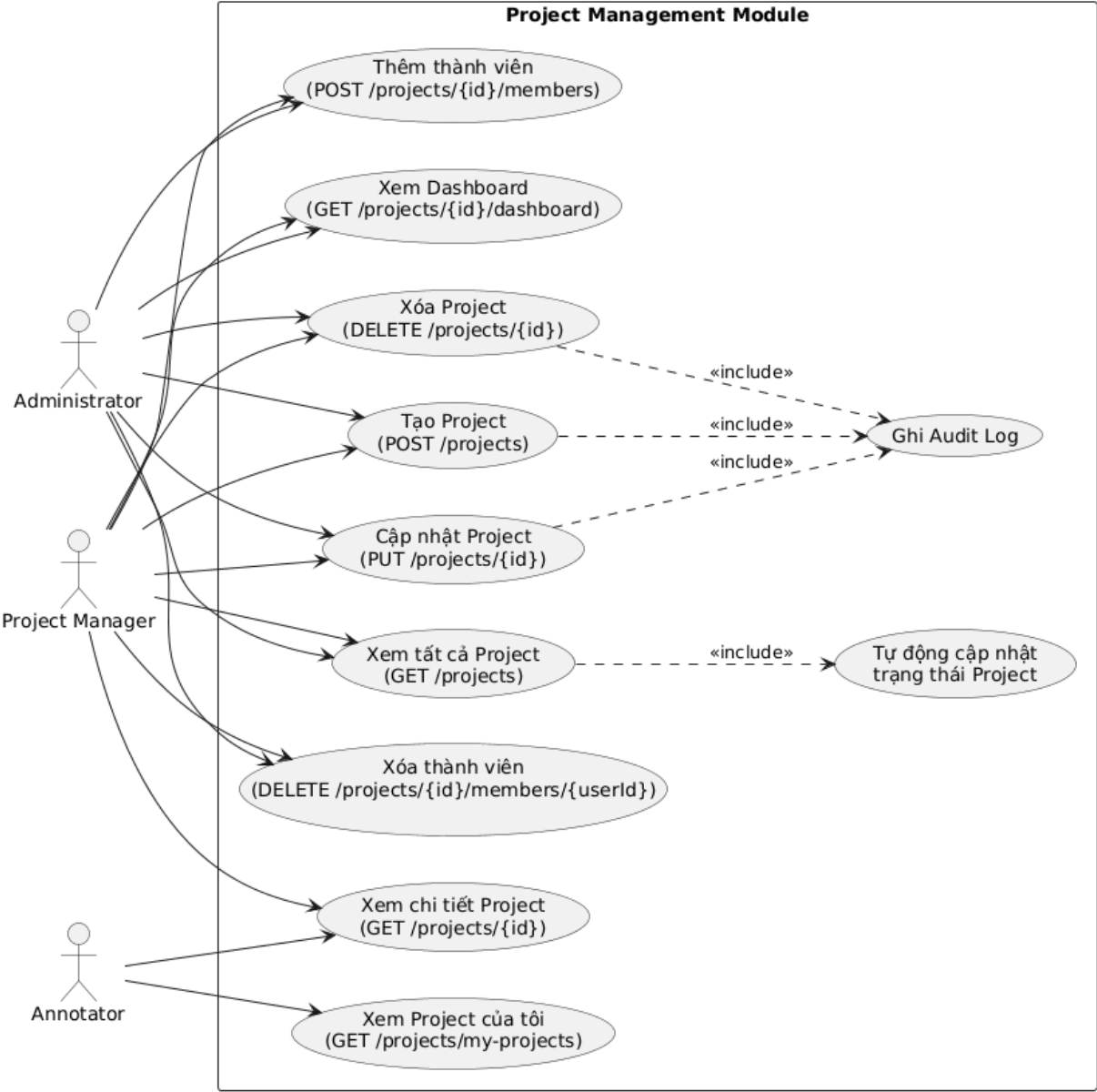
4.1.1 Use Case Diagram Tổng quan hệ thống



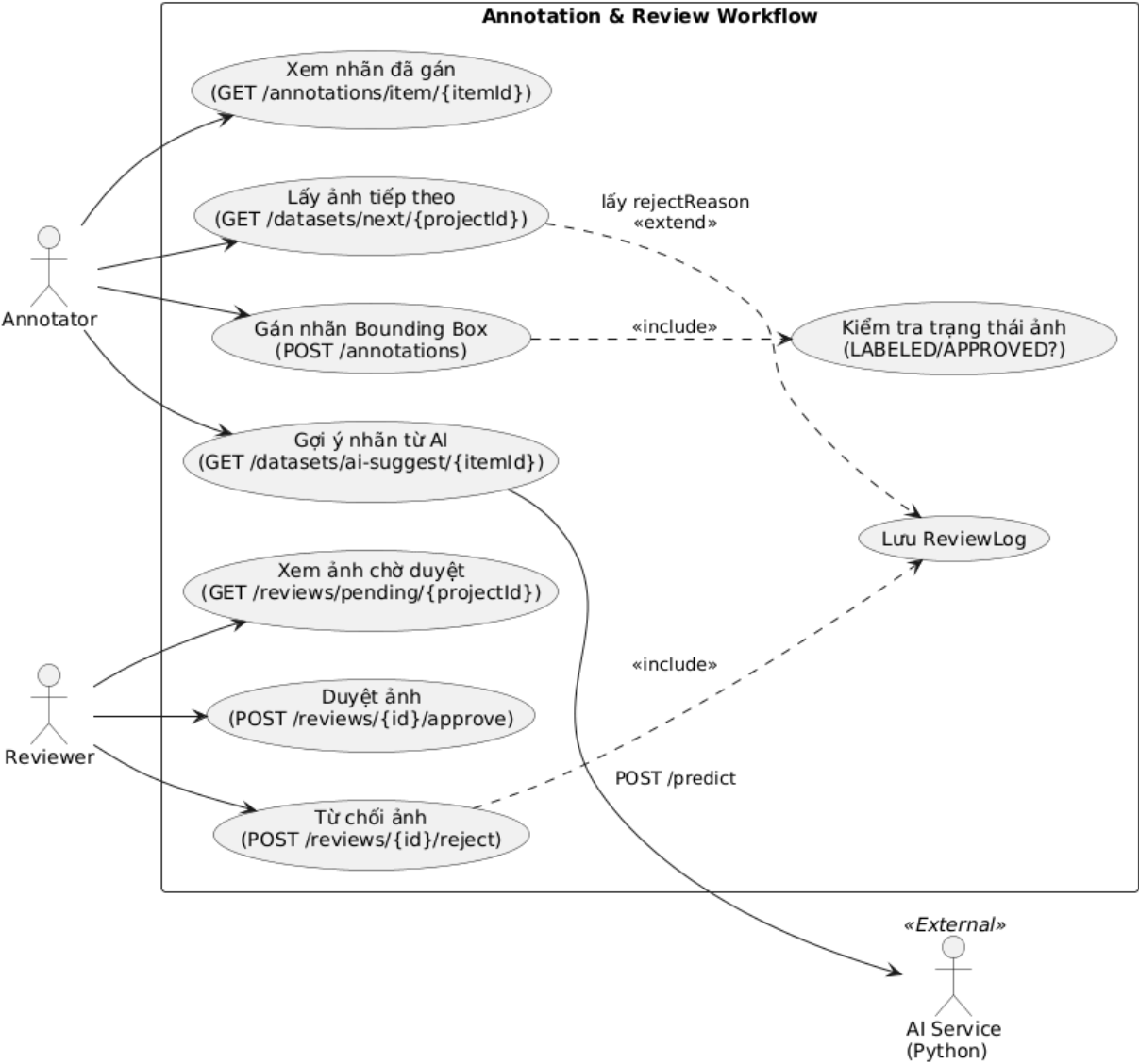
4.1.2 Use Case Diagram — Module Authentication (Chi tiết)



4.1.3 Use Case Diagram — Module Project Management (Chi tiết)



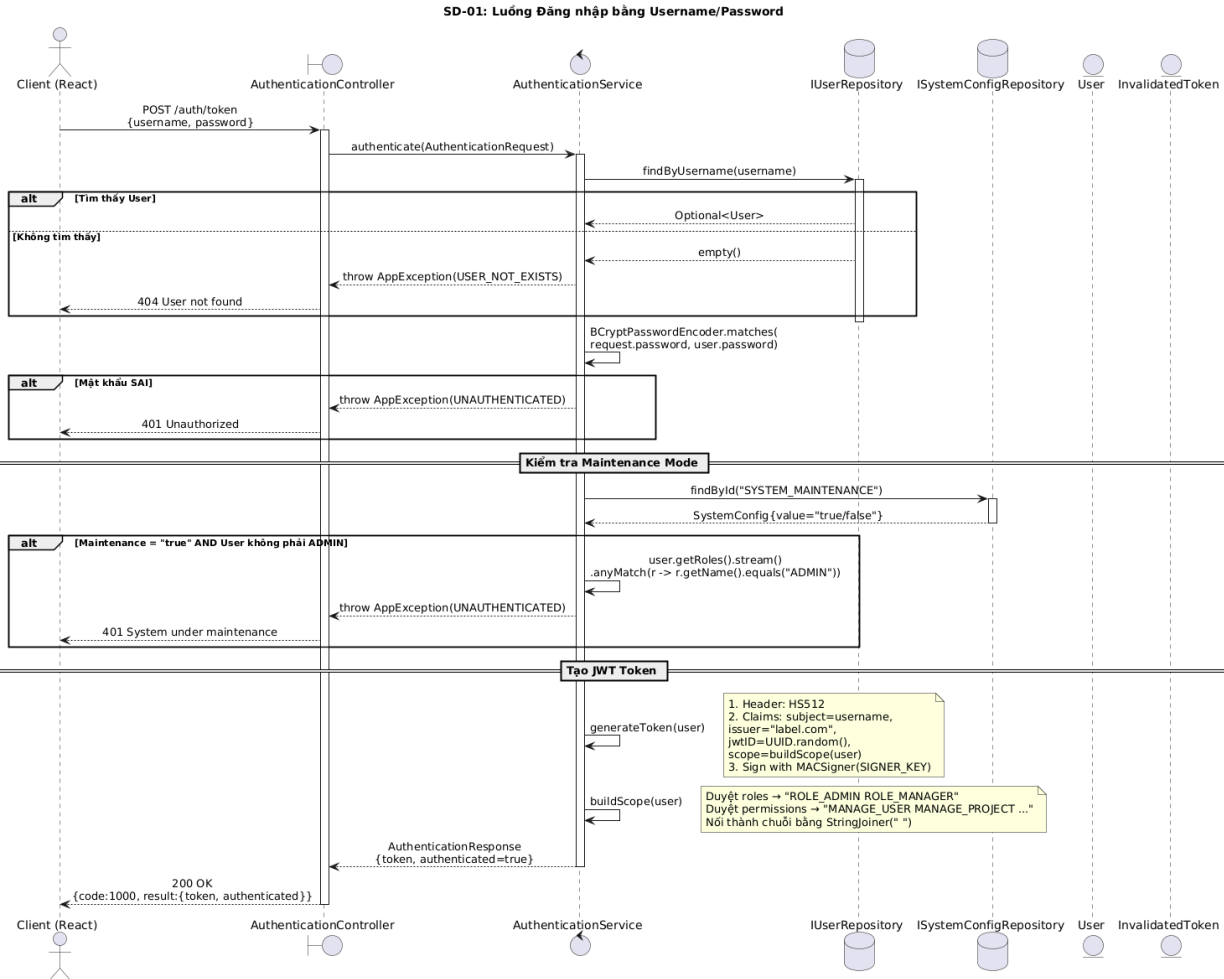
4.1.4 Use Case Diagram — Module Annotation & Review (Chi tiết)



4.2 Sequence Diagrams — Chi tiết từng luồng chức năng

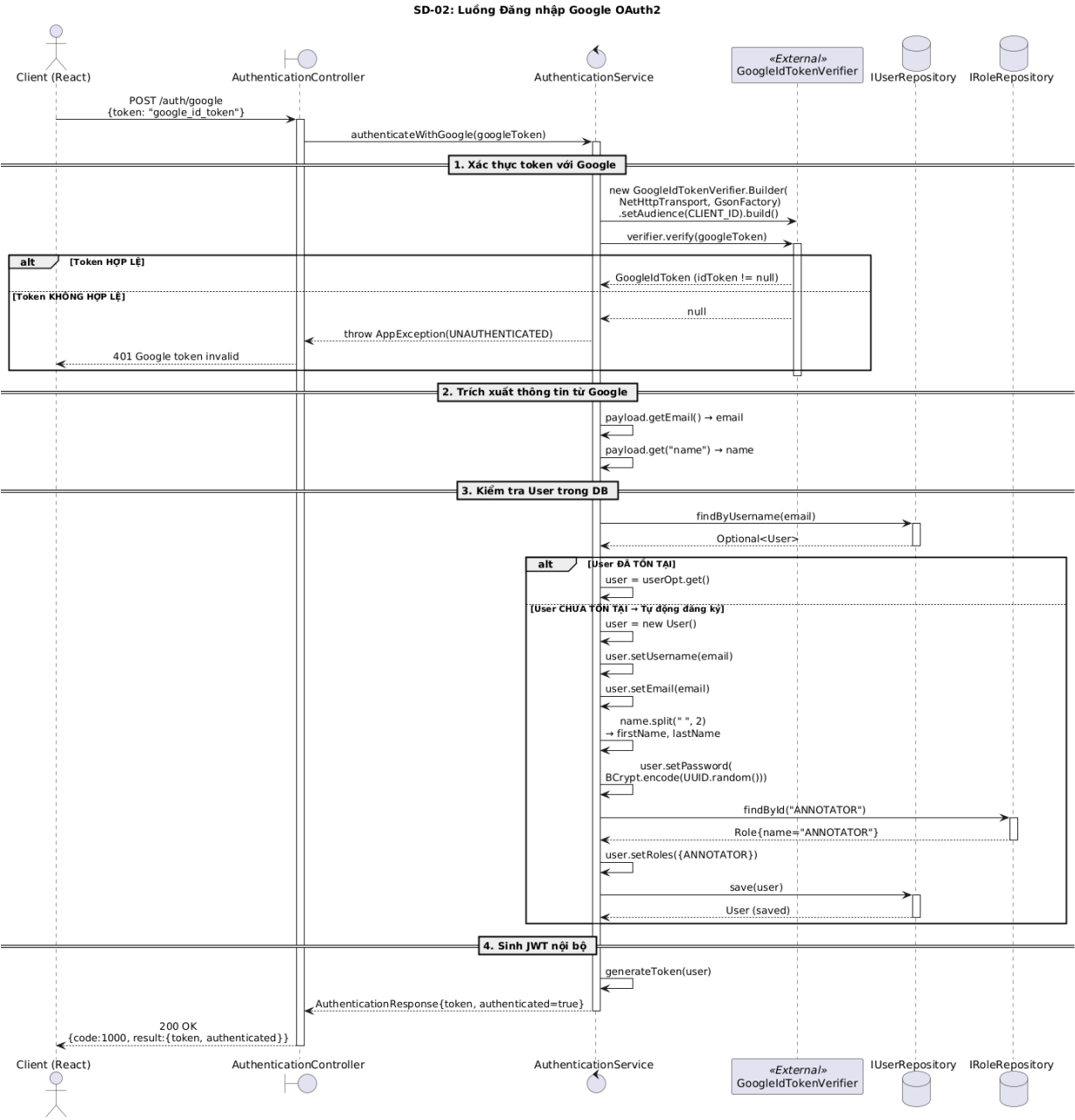
4.2.1 Sequence Diagram — Đăng nhập Local (POST /auth/token)

Luồng chính tại `AuthenticationController.authenticate()` → `AuthenticationService.authenticate()`



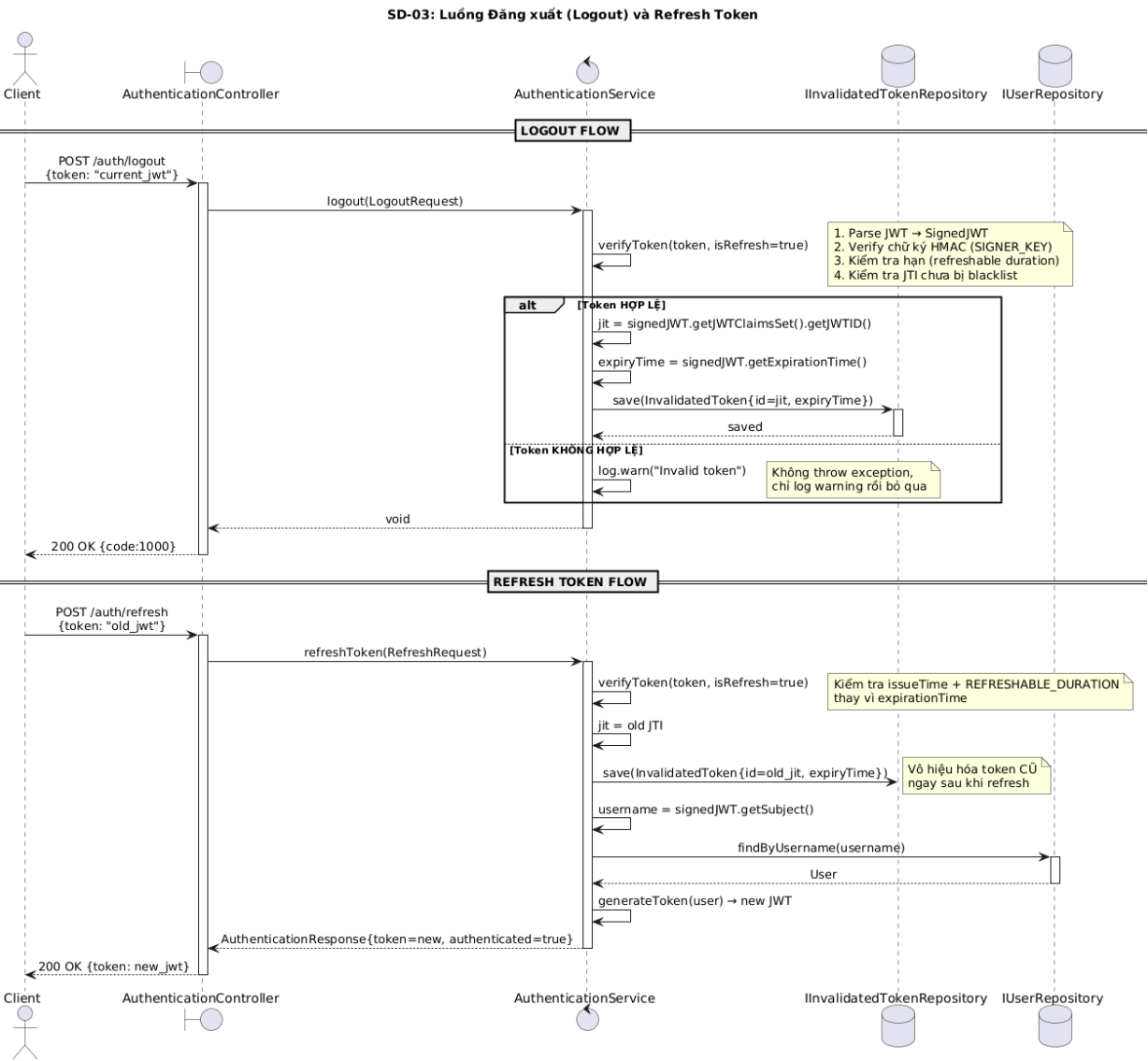
4.2.2 Sequence Diagram — Đăng nhập Google OAuth2 (POST /auth/google)

Luồng tại AuthenticationService.authenticateWithGoogle() .



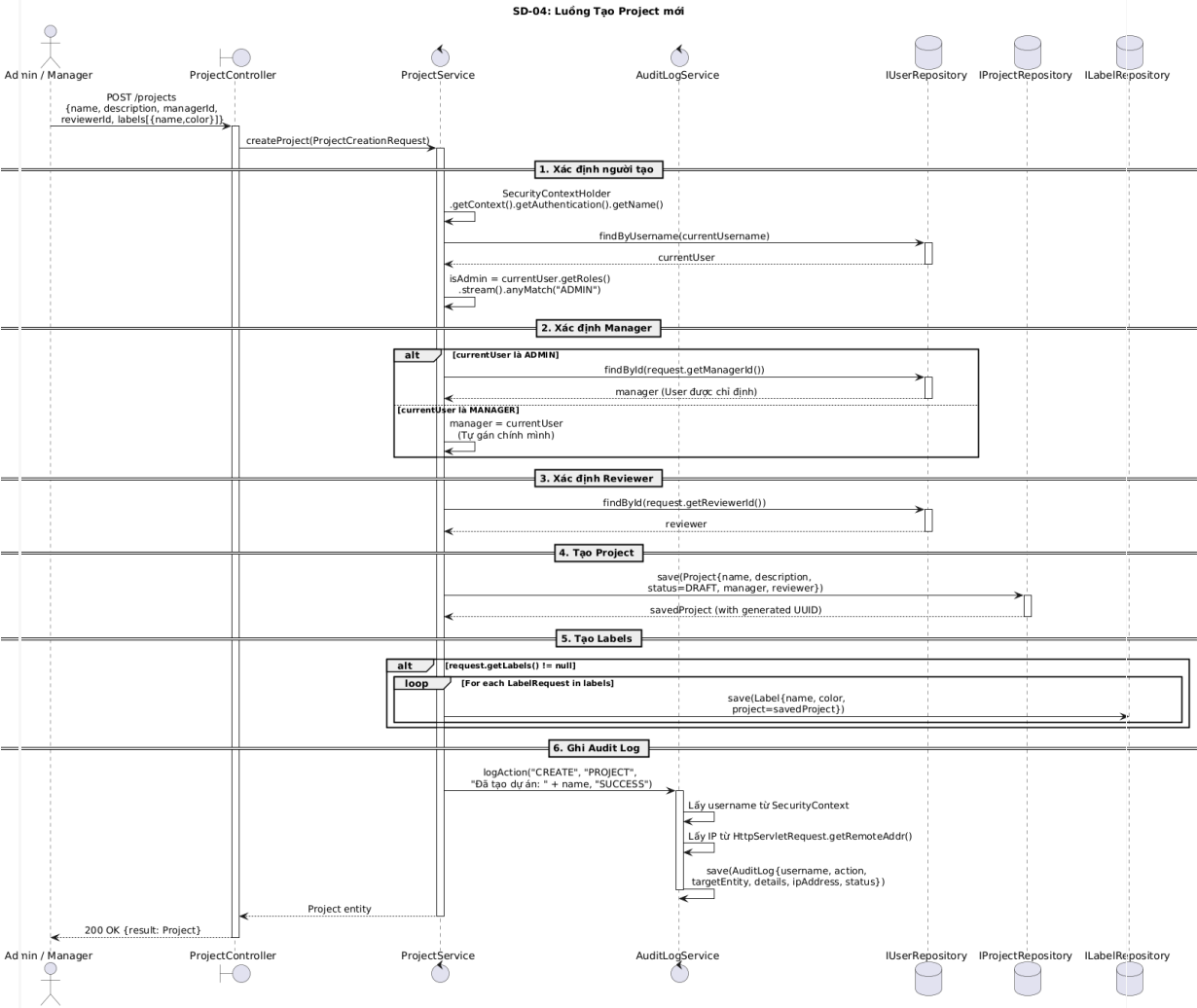
4.2.3 Sequence Diagram — Logout & Refresh Token

Luồng tại AuthenticationService.logout() và AuthenticationService.refreshToken() .



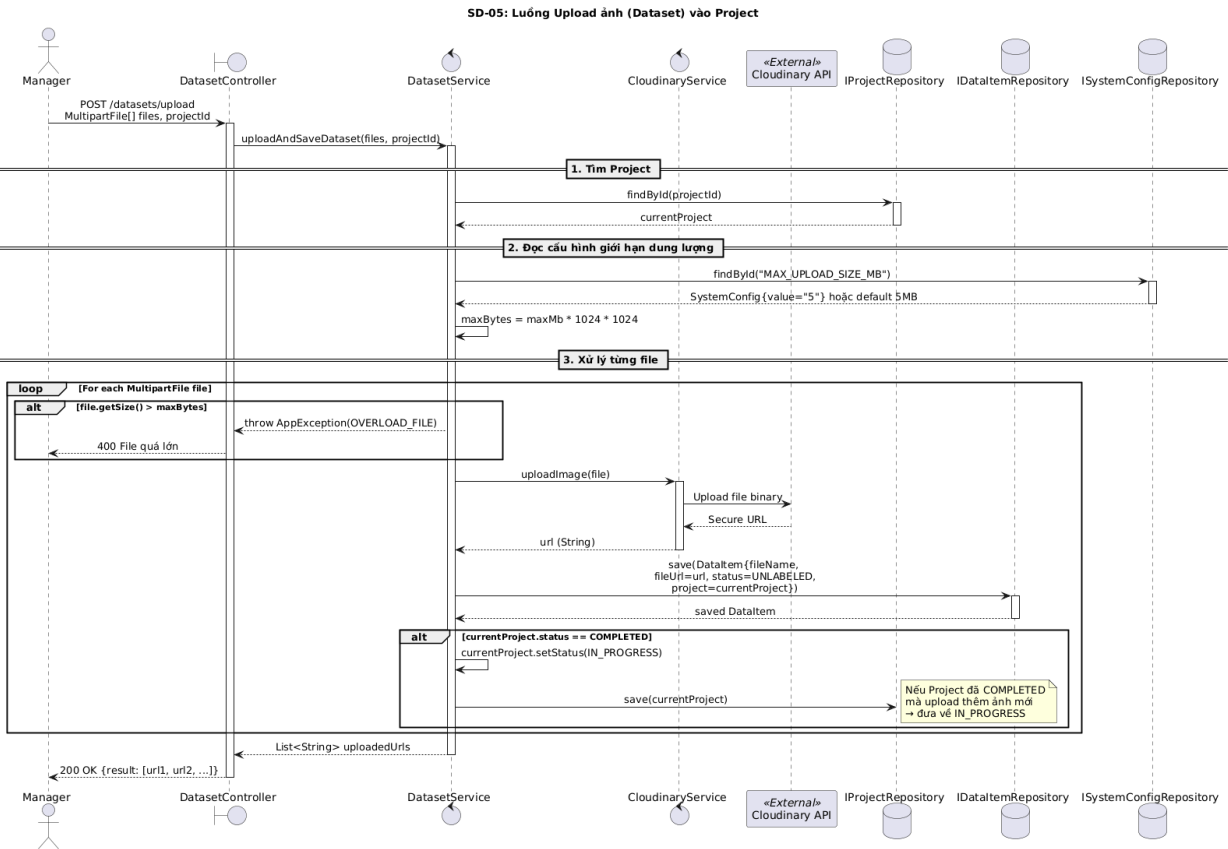
4.2.4 Sequence Diagram — Tạo Project mới (POST /projects)

Luồng tại `ProjectController.createProject()` → `ProjectService.createProject()` .



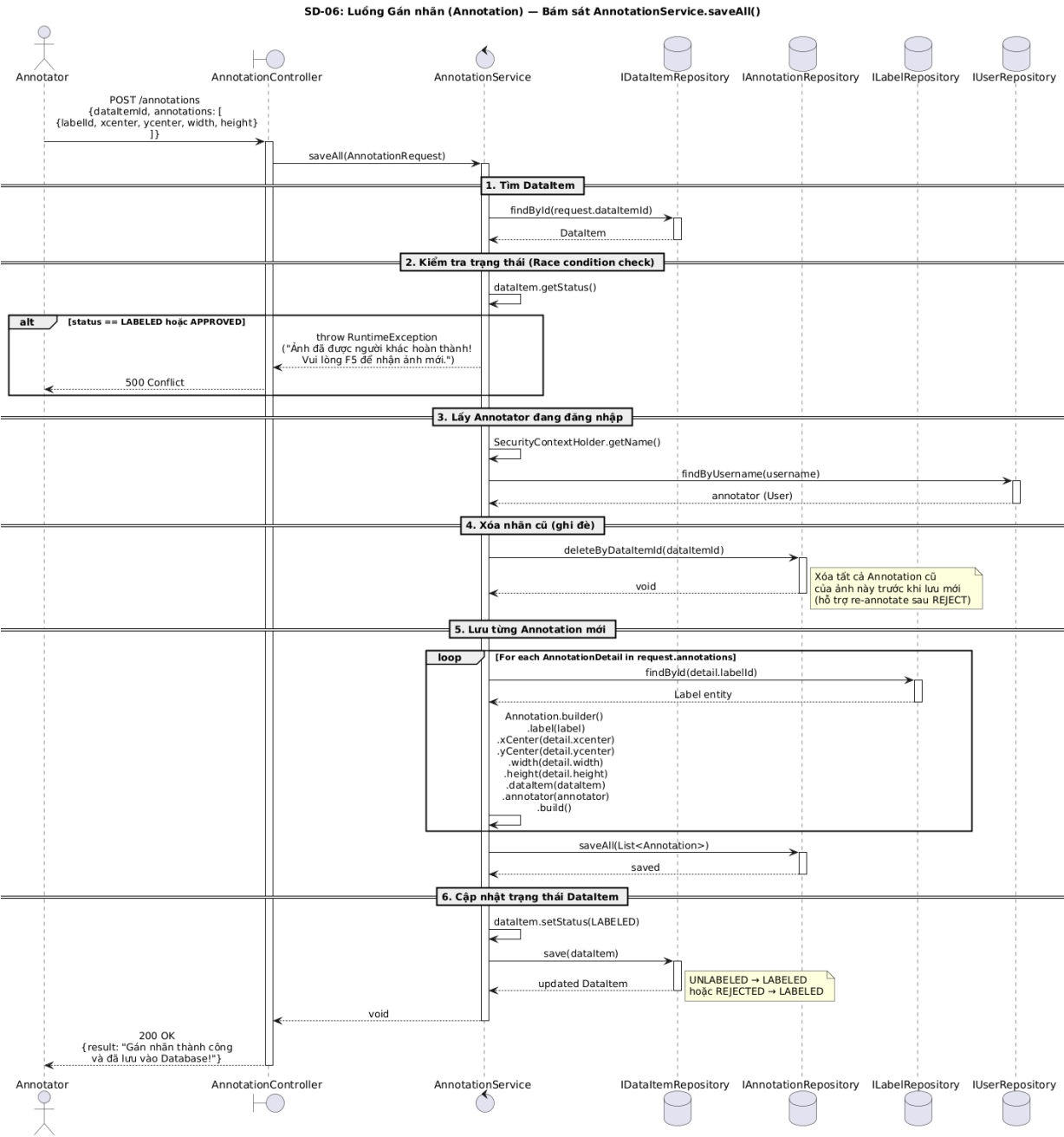
4.2.5 Sequence Diagram — Upload ảnh vào Project (POST /datasets/upload)

Luồng tại DatasetController.uploadDataset() → DatasetService.uploadAndSaveDataset()



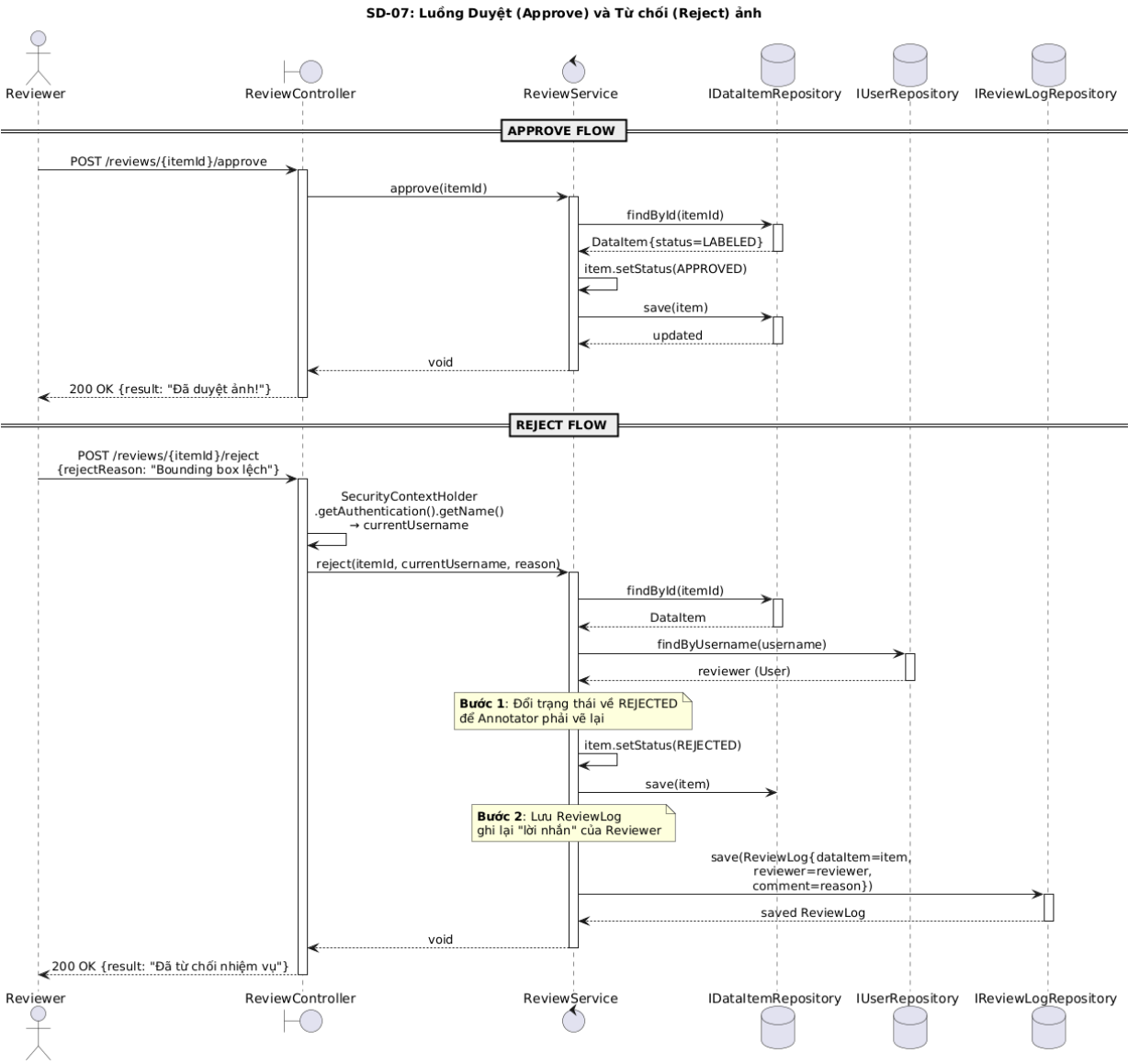
4.2.6 Sequence Diagram — Gán nhãn Annotation (POST /annotations)

Luồng tại AnnotationController.save() → AnnotationService.saveAll() .



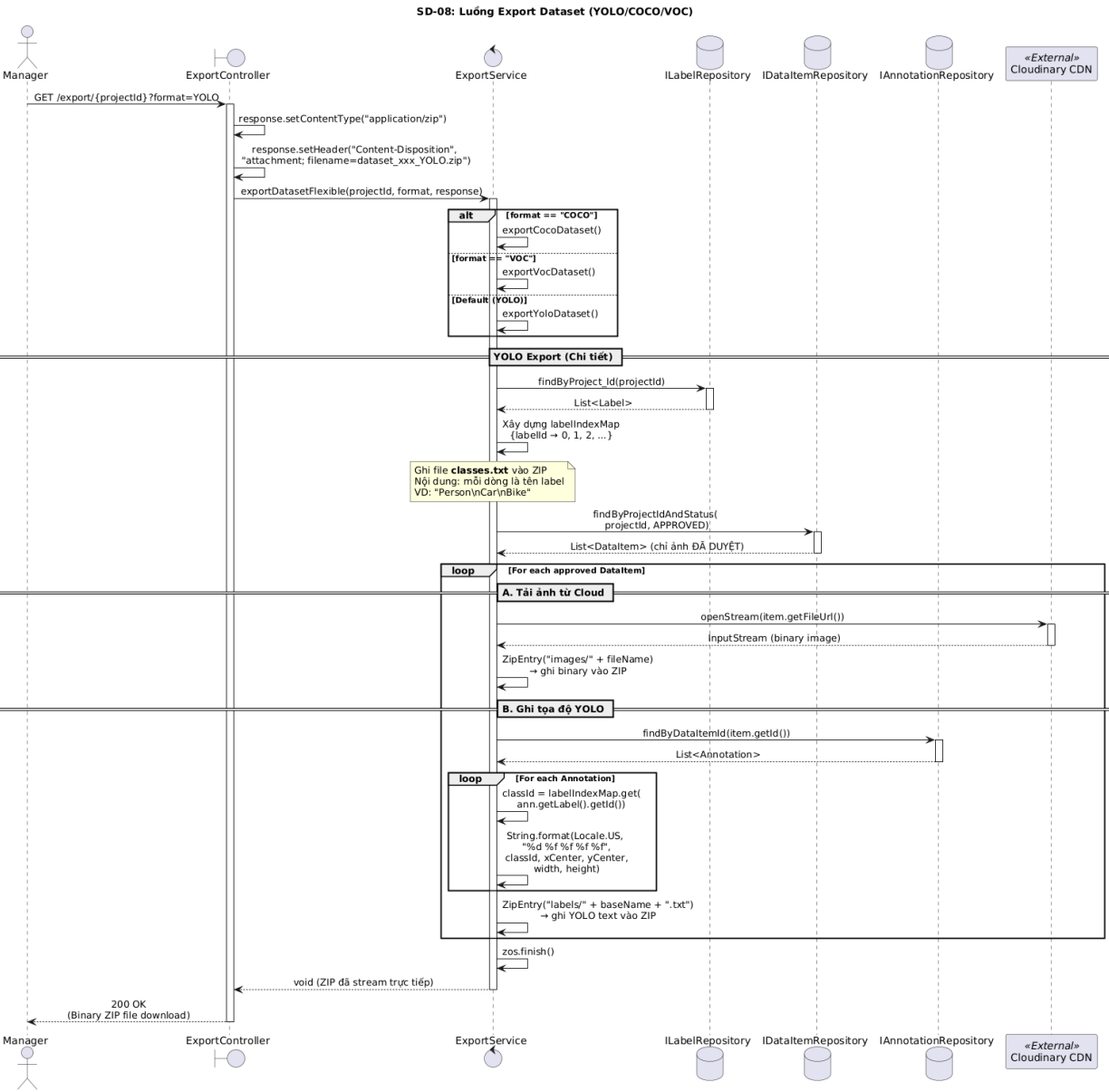
4.2.7 Sequence Diagram — Duyệt/Từ chối ảnh (Review Approve & Reject)

Luồng tại ReviewController → ReviewService .



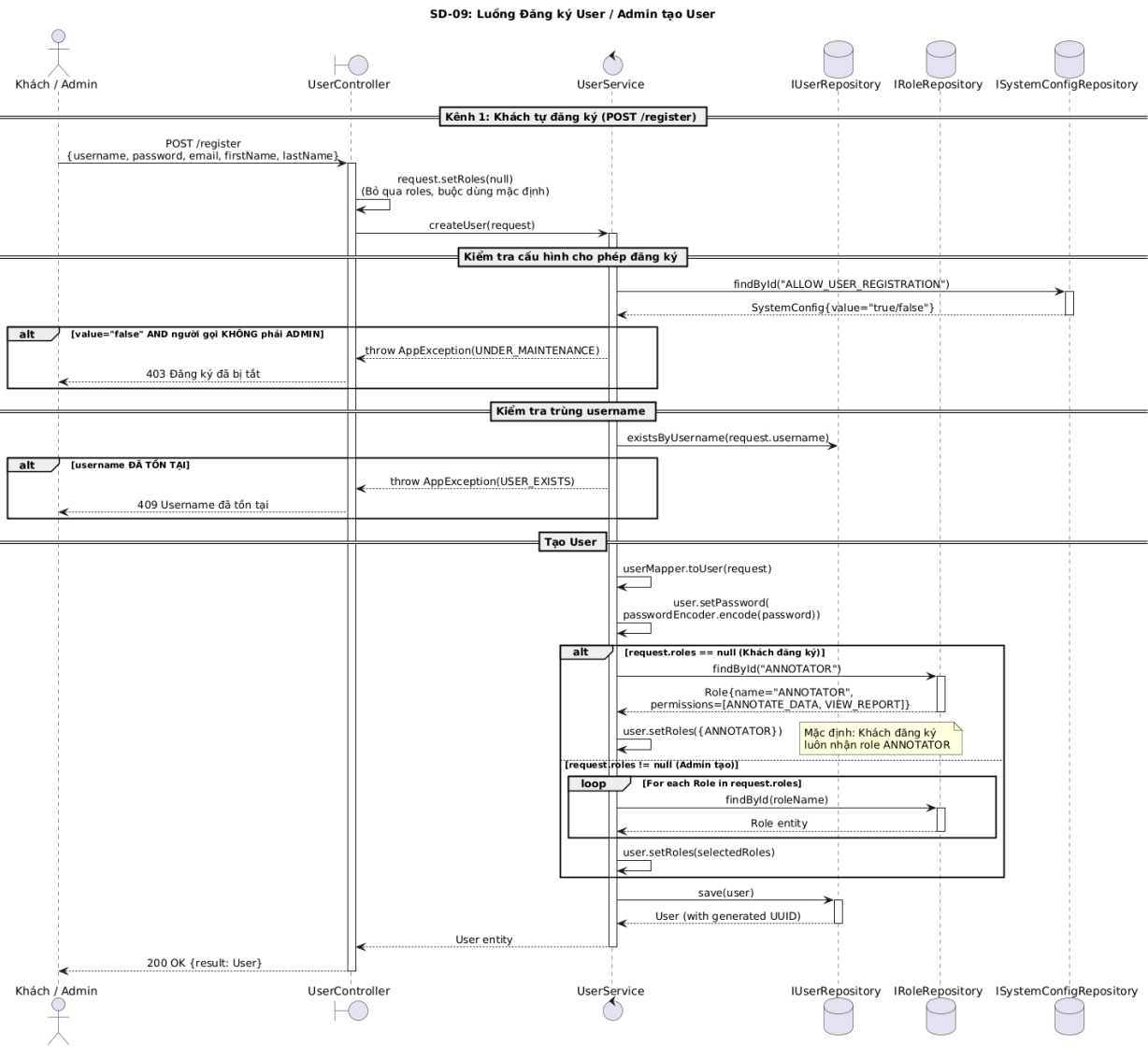
4.2.8 Sequence Diagram — Xuất Dataset (Export YOLO/COCO/VOC)

Luồng tại `ExportController` → `ExportService.exportDatasetFlexible()` .



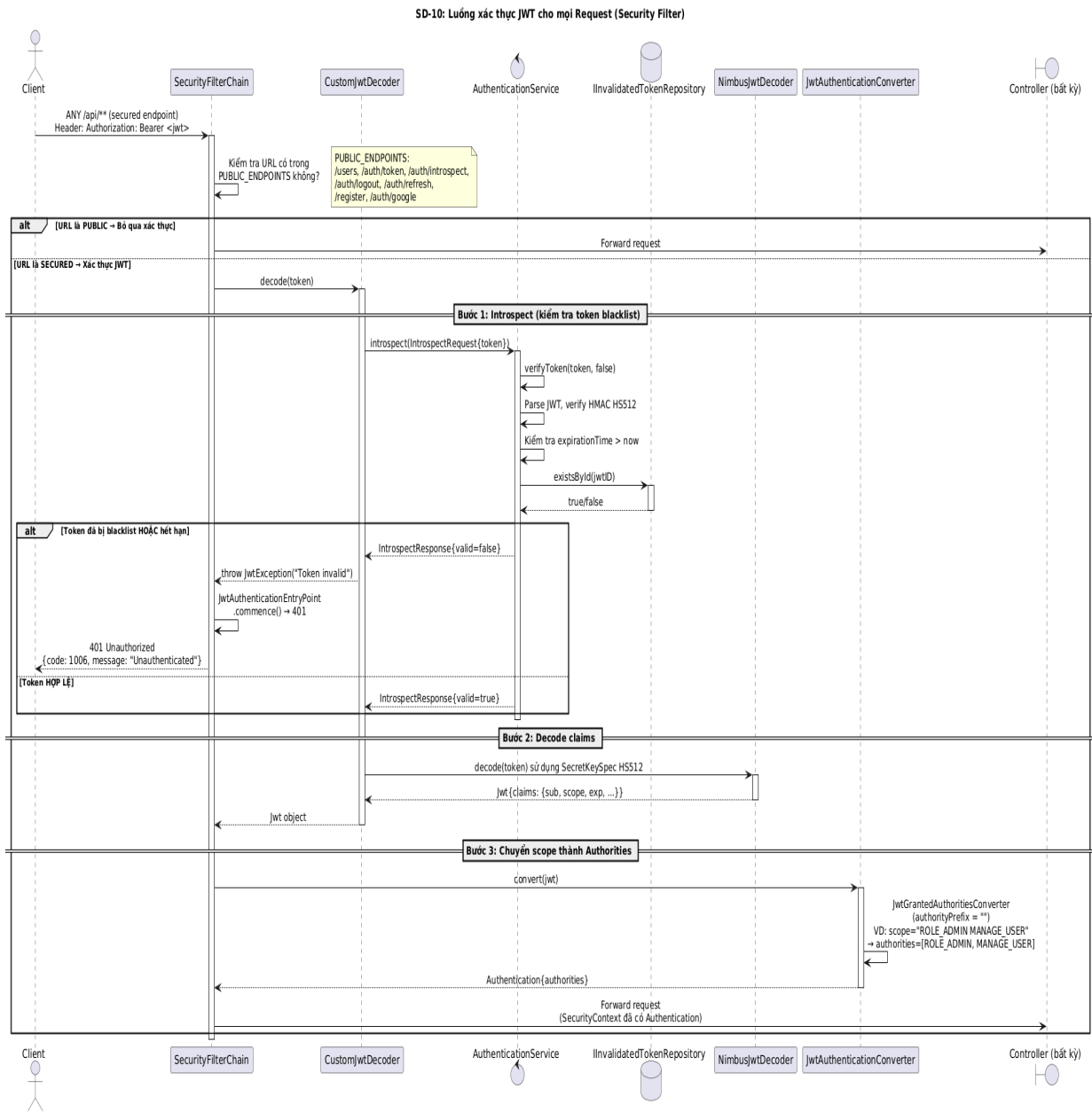
4.2.9 Sequence Diagram — Đăng ký User & Tạo User bởi Admin

Luồng tại `UserController.registerUser()` / `createUser()` → `UserService.createUser()` .



4.2.10 Sequence Diagram — JWT Validation (CustomJwtDecoder + SecurityFilterChain)

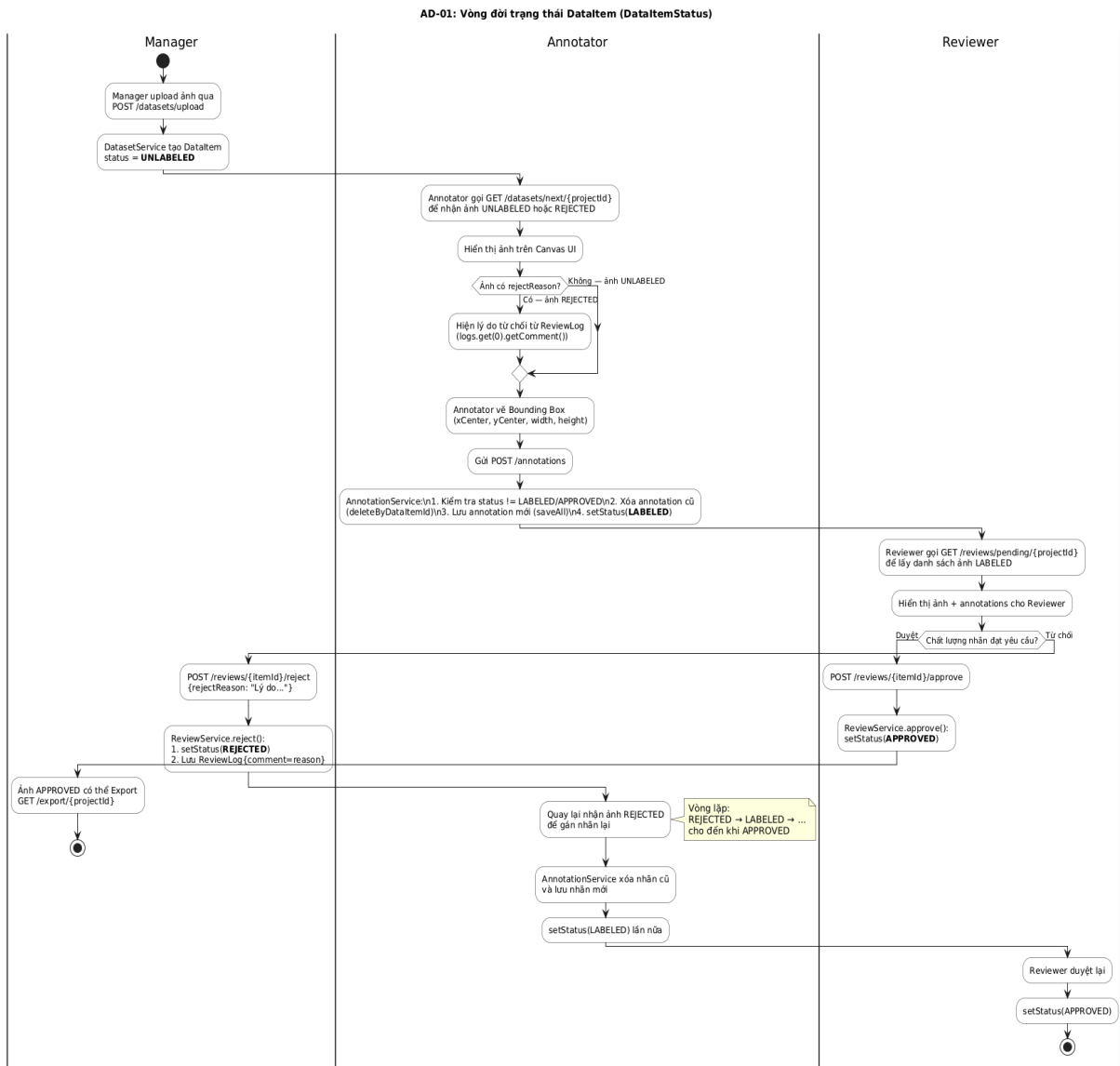
Luồng xác thực cho **mọi API request** có JWT.



4.3 Activity Diagrams

4.3.1 Activity Diagram — Vòng đời Dataltem (State Machine)

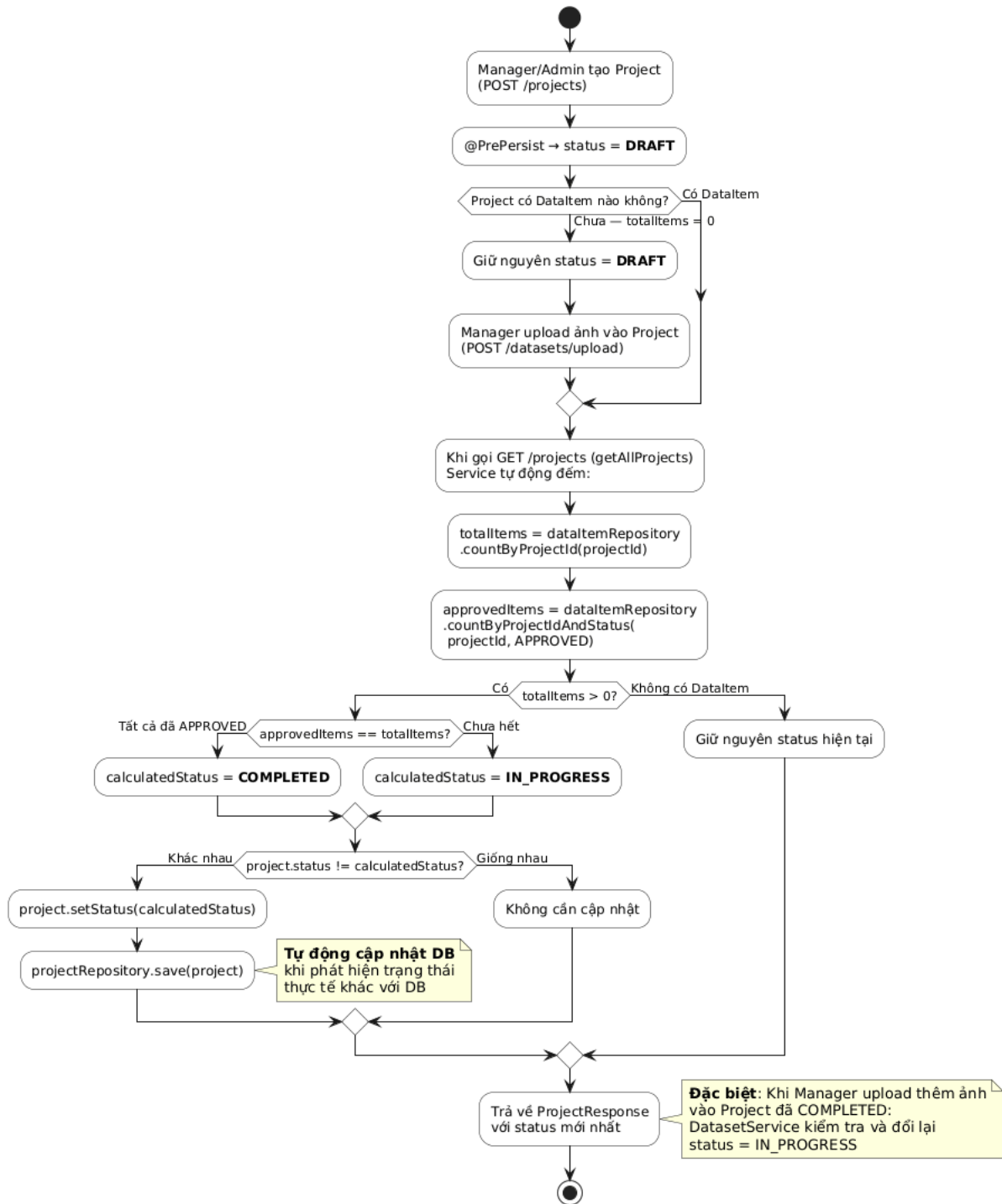
Dựa trên enum `DataItemStatus` = {UNLABELED, LABELED, APPROVED, REJECTED}.



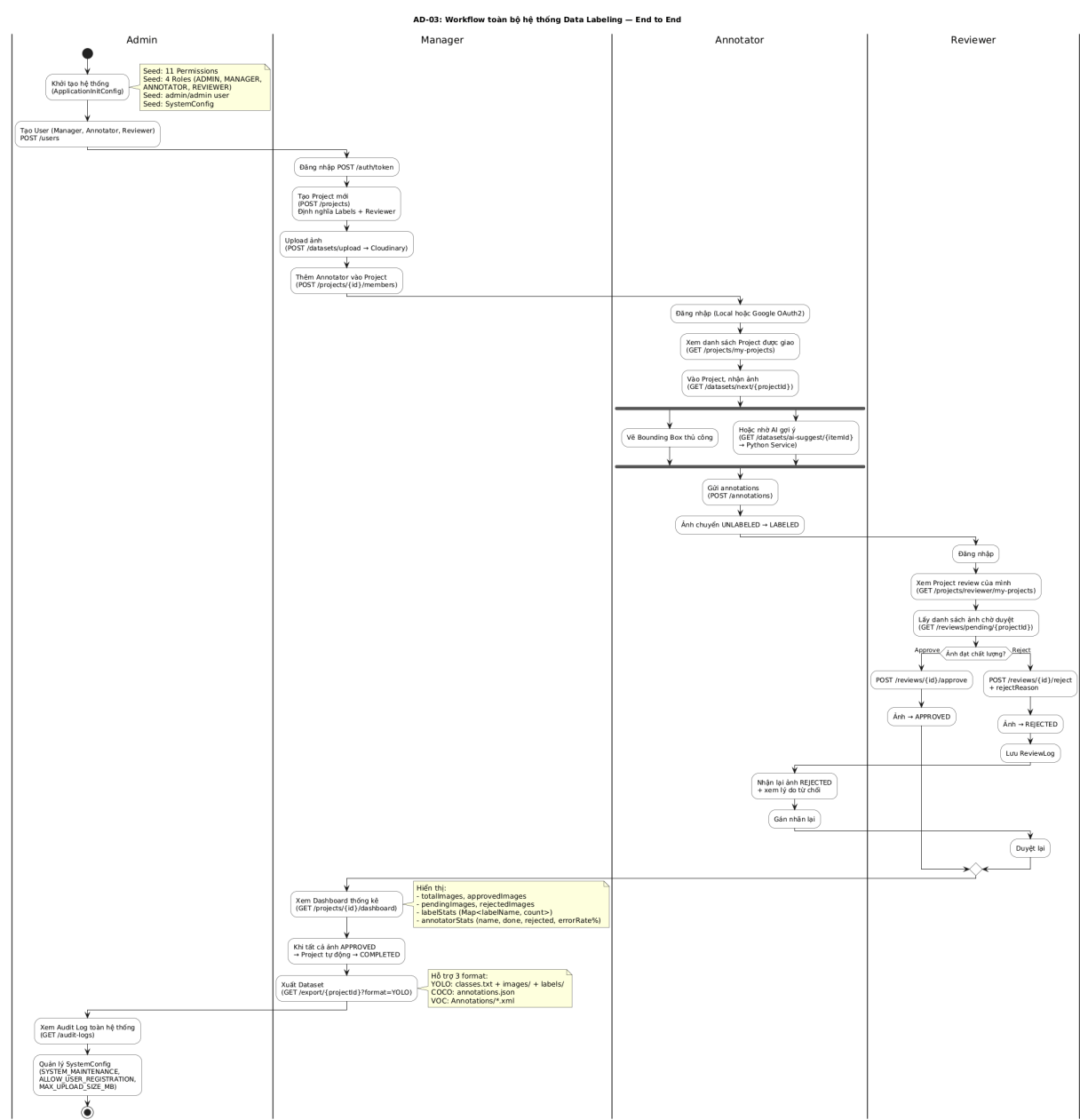
4.3.2 Activity Diagram — Vòng đời Project (ProjectStatus)

Dựa trên enum `ProjectStatus = {DRAFT, IN_PROGRESS, COMPLETED}` và logic tự động tính toán trong `ProjectService.getAllProjects()`

AD-02: Vòng đời trạng thái Project (ProjectStatus) — Tự động tính toán

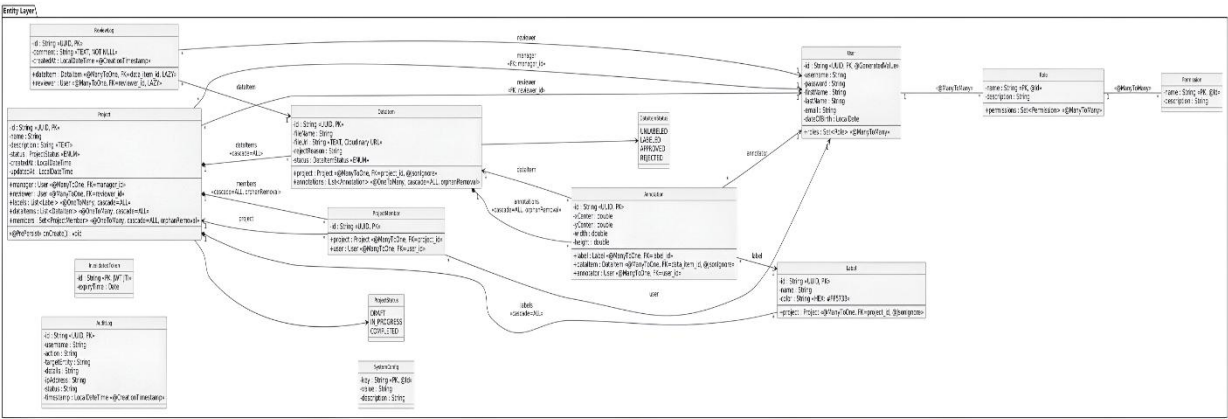


4.3.3 Activity Diagram — Toàn bộ Workflow hệ thống (End-to-End)

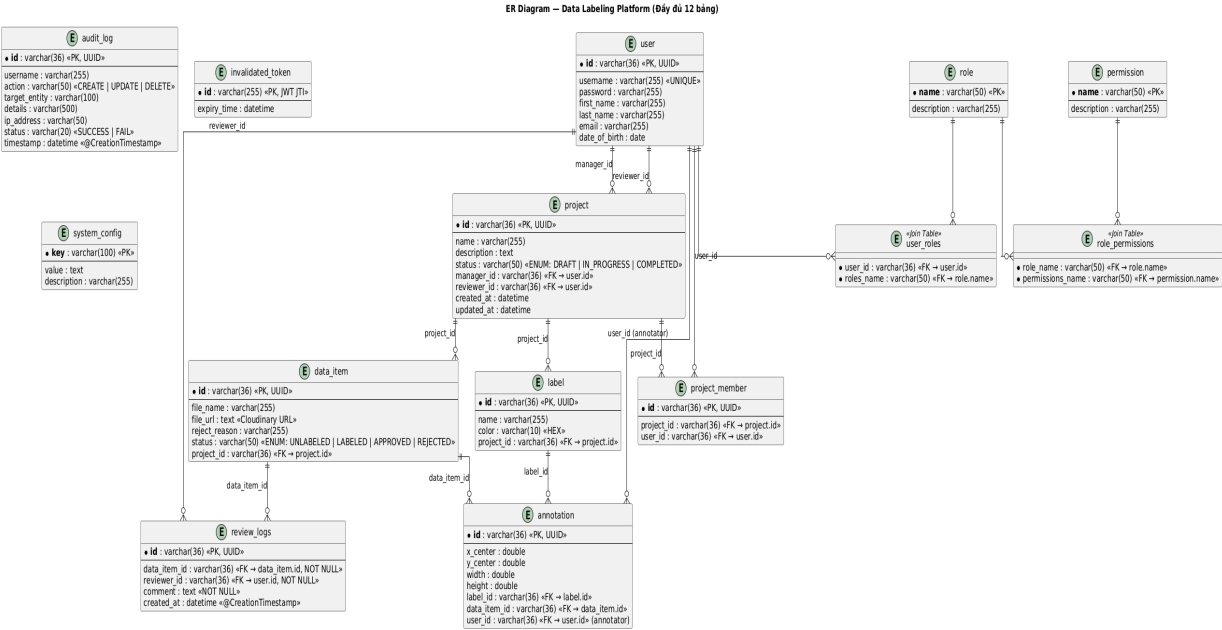


4.4 Class Diagram — Đầy đủ tất cả Entity, Attribute, Relationship

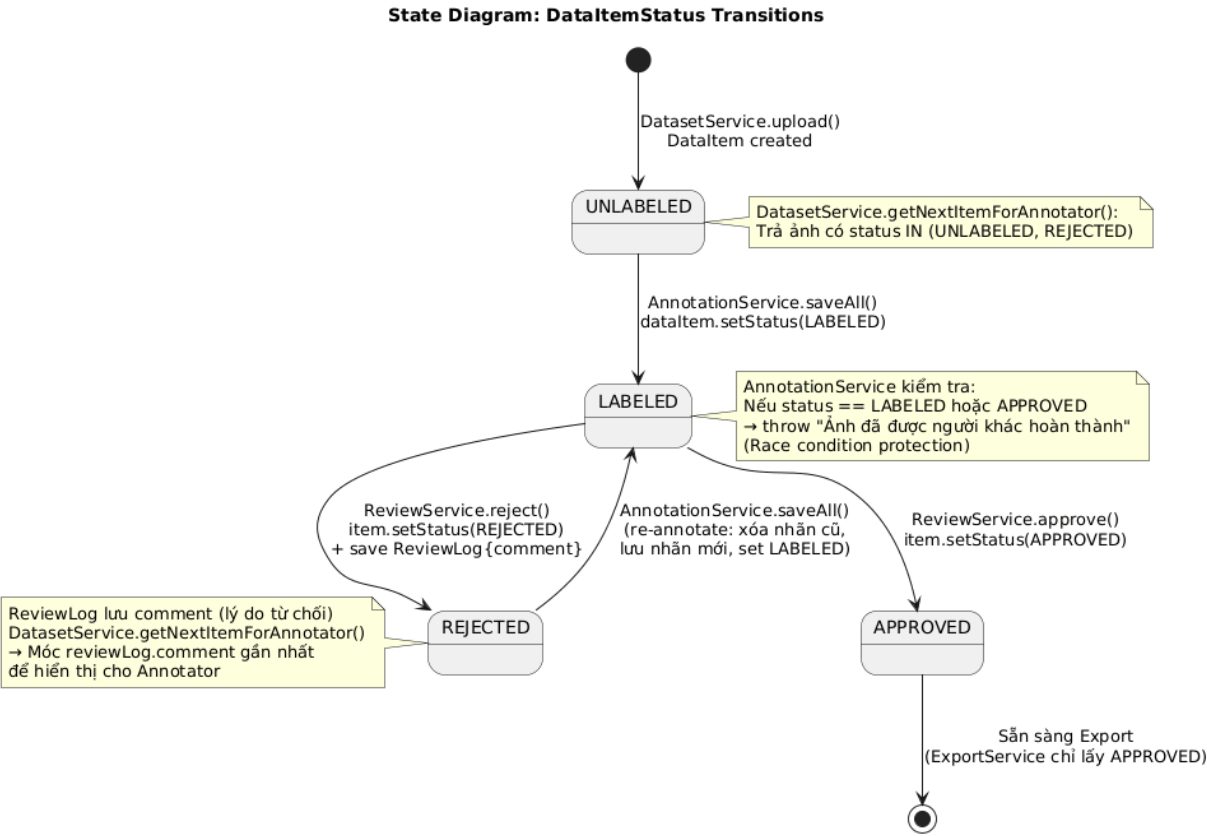
Bao gồm đầy đủ 12 Entity classes với tất cả attributes và relationships được trích xuất từ source code



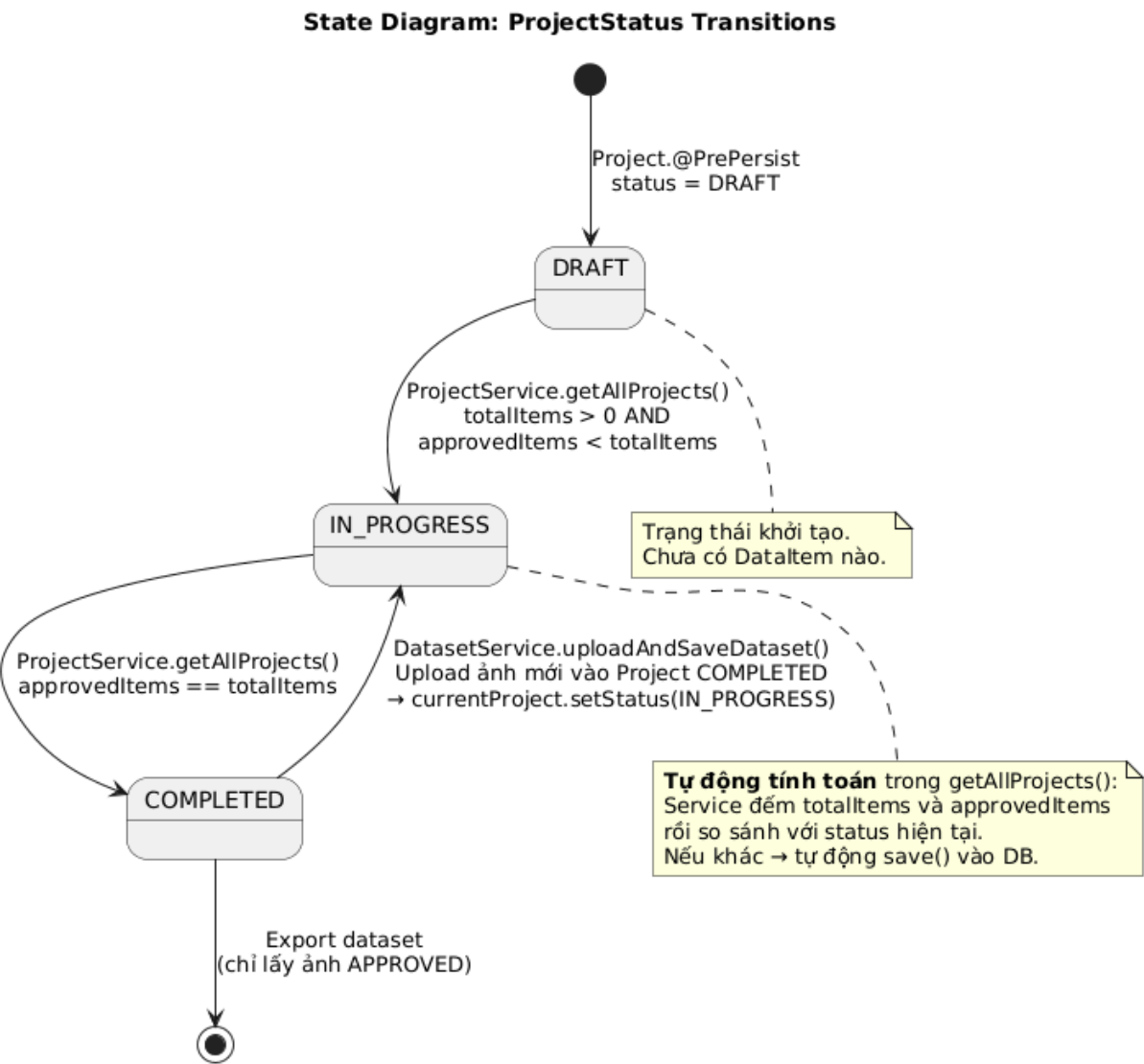
4.5 Entity-Relationship (ER) Diagram — Đầy đủ tất cả bảng và cột



4.5.1 State Diagram — DataItemStatus (Máy trạng thái)



4.5.2 State Diagram — ProjectStatus (Máy trạng thái)



5. Requirement Traceability Matrix (RTM)

Bảng RTM đảm bảo tất cả yêu cầu được ánh xạ tới Use Case, API endpoint, DB Entity, và Diagram PlantUML tương ứng.

Req ID	Mô tả yêu cầu	Use Case	API Endpoint	DB Entity	Diagram
FR-01a	Đăng nhập Local	UC-01	POST /auth/token	User, InvalidatedToken	SD-01
FR-01b	Đăng nhập Google OAuth2	UC-02	POST /auth/google	User, Role	SD-02
FR-01c	Đăng xuất (Logout)	UC-03	POST /auth/logout	InvalidatedToken	SD-03
FR-01d	Refresh Token	UC-04	POST /auth/refresh	InvalidatedToken	SD-03
FR-01e	Introspect Token	UC-05	POST /auth/introspect	InvalidatedToken	SD-10
FR-01f	Đăng ký tài khoản	UC-06	POST /register	User, Role	SD-09
FR-02a	Tạo User (Admin)	UC-07	POST /users	User, Role	SD-09
FR-02b	Xem danh sách User	UC-08	GET /users	User	-
FR-02c	Cập nhật User	UC-09	PUT /users/{id}	User, Role	-
FR-02d	Xóa User	UC-10	DELETE /users/{id}	User	-
FR-02e	Xem thông tin cá nhân	UC-11	GET /users/myInfor	User	-
FR-03a	Tạo Project mới	UC-12	POST /projects	Project, Label, AuditLog	SD-04
FR-03b	Xem danh sách Project	UC-13	GET /projects	Project, Dataltem	AD-02
FR-03c	Cập nhật Project	UC-14	PUT /projects/{id}	Project, AuditLog	-
FR-03d	Xóa Project	UC-15	DELETE /projects/{id}	Project, AuditLog	-

FR-03e	Thêm thành viên	UC-16	POST /projects/{id}/members	ProjectMember	SD-04
FR-03f	Xóa thành viên	UC-17	DELETE /projects/{id}/members/{userId}	ProjectMember	-
FR-03g	Dashboard thống kê	UC-18	GET /projects/{id}/dashboard	DataItem, Annotation	AD-03
FR-03h	Xem Project được giao	UC-19	GET /projects/my-projects	Project, ProjectMember	AD-03
FR-04a	Upload ảnh	UC-20	POST /datasets/upload	DataItem, Project	SD-05
FR-04b	Xem danh sách ảnh	UC-21	GET /datasets/project/{id}	DataItem, ReviewLog	-
FR-04c	Xóa ảnh	UC-22	DELETE /datasets/{id}	DataItem, AuditLog	-
FR-04d	Lấy ảnh tiếp theo	UC-23	GET /datasets/next/{projectId}	DataItem, ReviewLog	AD-01
FR-04e	Gợi ý AI	UC-24	GET /datasets/ai-suggest/{itemId}	DataItem	AD-03
FR-05a	Gán nhãn Bounding Box	UC-25	POST /annotations	Annotation, DataItem, Label	SD-06
FR-05b	Xem nhãn đã gán	UC-26	GET /annotations/item/{itemId}	Annotation	-
FR-06a	Xem ảnh chờ duyệt	UC-27	GET /reviews/pending/{projectId}	DataItem	SD-07
FR-06b	Duyệt ảnh (Approve)	UC-28	POST /reviews/{id}/approve	DataItem	SD-07
FR-06c	Từ chối ảnh (Reject)	UC-29	POST /reviews/{id}/reject	DataItem, ReviewLog	SD-07
FR-06d	Xem Project review	UC-30	GET /projects/reviewer/my-projects	Project	AD-03
FR-07a	Xuất Dataset YOLO	UC-31	GET /export/{projectId}?format=YOLO	DataItem, Annotation, Label	SD-08
FR-07b	Xuất Dataset COCO	UC-32	GET /export/{projectId}?format=COCO	DataItem, Annotation, Label	SD-08
FR-07c	Xuất Dataset VOC	UC-33	GET /export/{projectId}?format=VOC	DataItem, Annotation, Label	SD-08
FR-08a	Quản lý Role	UC-34	POST/GET/DELETE /roles	Role, Permission	-

FR-08b	Quản lý Permission	UC-35	POST/GET/DELETE /permissions	Permission	-
FR-08c	Cấu hình hệ thống	UC-36	POST/GET /system-configs	SystemConfig	SD-01
FR-08d	Xem Audit Log	UC-37	GET /audit-logs	AuditLog	AD-03
NFR-01	JWT Security (HS512)	UC-01~05	SecurityFilterChain	InvalidatedToken	SD-10
NFR-02	RBAC Authorization	All	@PreAuthorize	Role, Permission	SD-10
NFR-03	Password Hashing	UC-06/07	UserService	User	SD-09
NFR-04	Maintenance Mode	UC-01/06	AuthService/UserService	SystemConfig	SD-01
NFR-05	File Size Limit	UC-20	DatasetService	SystemConfig	SD-05

End of Document